

Informe: Creación de Hoppers

Descripción: heurística

La forma en la que se definió la función heurística hace que se estime qué jugador tiene ventaja cuando Minimax alcanza el límite de profundidad sin encontrar un estado terminal. La evaluación se calcula desde la perspectiva de P1: los valores positivos favorecen a P1, los negativos favorecen a P2 y los valores cercanos a cero indican una posición equilibrada, donde el rango es de -1 a 1.

La heurística combina tiene dos componentes:

- Progreso general, con peso de 0.60: mide cuánto han avanzado las piezas de cada jugador hacia la esquina contraria. El progreso individual se normaliza entre 0 y 1, donde 0 representa la esquina inicial y 1 la esquina objetivo. Entonces si una pieza de P1 se encuentra en la fila 9, columna 9, su progreso será de 1.
- Ocupación del campamento objetivo, con peso de 0.40: compara cuántas piezas propias tiene cada jugador dentro del campamento al que debe llegar. También lo normalizo sobre 15 al restar piezas de P1 en el campamento contrario menos piezas de P2 en el campamento contrario.

La fórmula utilizada es:

```
heuristic = (  
    0.60 * progress_balance  
    + 0.40 * camp_balance  
)
```

El progreso tiene un peso mayor porque permite distinguir posiciones favorables desde las primeras etapas de la partida, cuando todavía no hay piezas dentro del campamento contrario. La ocupación del objetivo recibe una bonificación adicional porque acercarse a completar el campamento representa una ventaja más directa.

Si el estado ya es terminal, entonces la heurística no realiza una estimación. La función hace una validación antes de seguir y en ese caso se utiliza `utility()`, que devuelve +1 si gana P1 y -1 si gana P2. Mientras el juego sigue sin ganadores su valor es de 0.

Una limitación de esta heurística es que el progreso total puede considerar equivalentes dos situaciones distintas: avanzar mucho una sola pieza o avanzar poco varias piezas porque avanzar una sola pieza tendría más peso relativo que el avance que múltiples piezas tienen. Si ambas opciones producen la misma suma de distancias, reciben el mismo valor.

Manejo de la profundidad

La búsqueda utiliza un parámetro configurable llamado *depth*, que indica cuántos turnos futuros puede explorar Minimax. Cada llamada recursiva reduce este valor en uno.

Por ejemplo, con profundidad 3 y siendo el turno de P1, se consideran:

1. Una jugada de P1.
2. Una posible respuesta de P2 a esa jugada de P1.
3. Una nueva jugada de P1.

Cuando *depth* llega a cero, la búsqueda se detiene y utiliza la función heurística para evaluar el estado alcanzado.

```
if terminal(state):
```

```
    return float(utility(state))
```

Luego, si esto no se cumple:

```
if depth == 0:
```

```
    return heuristic(state)
```

La comprobación del estado terminal se realiza primero porque una victoria real debe tener prioridad sobre cualquier estimación heurística. Esto lo tiene que hacer para P1 y para P2 la primera jugada tiene 32 posibles movimientos legales, así que se visitan 32 nodos en la primera profundidad o nivel.

Aumentar la profundidad permite anticipar más movimientos, pero también incrementa considerablemente la cantidad de estados visitados. En las pruebas realizadas desde el tablero inicial, la profundidad 1 visitó 32 nodos, mientras que la profundidad 2 visitó 234. La poda alfa-beta redujo parte de este trabajo descartando ramas que ya no podían mejorar la decisión, que fueron más de 32 podas.

Manejo del tiempo

El agente tiene un límite máximo de 20 segundos por jugada. Para aprovechar ese tiempo se utiliza profundidad iterativa: primero se ejecuta una búsqueda completa con profundidad 1, después con profundidad 2, luego con profundidad 3 y así sucesivamente. Lo puedo dejar pensando más tiempo, pero en la vida real considero que esto es el tiempo ideal de respuesta.

Antes de comenzar se calcula un instante límite o deadline mediante `time.perf_counter()`. Todas las llamadas recursivas reciben el mismo límite y comprueban periódicamente si todavía queda tiempo.

Cuando se alcanza el límite, se lanza una excepción `SearchTimeout`. Esta excepción interrumpe inmediatamente la profundidad que se estaba explorando y permite devolver la acción encontrada en la última profundidad completada.

Por ejemplo, si la profundidad 3 terminó correctamente, pero el tiempo se agotó durante la profundidad 4, el agente devuelve el resultado de profundidad 3. No se utiliza el resultado parcial de profundidad 4 porque todavía podrían existir acciones sin evaluar que fueran mejores.

También se conserva una jugada legal de respaldo por si el tiempo termina antes de completar la primera profundidad. Además, se utiliza un pequeño margen de seguridad para evitar superar externamente los 30 segundos.

Observaciones

La representación de las acciones conserva la ruta completa. Una acción como:

((8, 9), (6, 7), (6, 5), (4, 5))

representa un solo turno con tres saltos encadenados. En la interfaz esto se puede ver como la última jugada. Esto permite validar cada parte de la ruta y mostrar claramente los saltos realizados.

Las pruebas verificaron movimientos simples, saltos sobre piezas propias y rivales, cadenas de saltos, inmutabilidad de los estados, cambio de turno, acciones ilegales, victorias y la regla anti-bloqueo, que según recuerdo quedaba a criterio de cada quién. También se comprobó que Minimax devolviese acciones legales y que el controlador termina inmediatamente cuando detecta un ganador.

La profundidad iterativa produjo resultados distintos según el tiempo disponible. Con un límite de un segundo, el agente logró completar varias profundidades y devolvió una jugada legal sin dejar de responder. Un mayor tiempo permite analizar más niveles, aunque no garantiza necesariamente una jugada diferente.

Las partidas de agente contra agente pueden ser largas porque Hoppers permite mover las piezas en varias direcciones, incluyendo movimientos hacia atrás. Esto puede provocar posiciones repetidas si la heurística considera que esos movimientos tienen valores similares. Sin embargo, el controlador permite establecer un límite de turnos durante las pruebas y siempre puede cerrarse desde la interfaz.

Finalmente, se agregó una interfaz con Pygame que permite jugar humano contra agente y observar agente contra agente. La lógica del juego se mantiene separada de la interfaz, por lo que las mismas funciones pueden utilizarse desde la consola, las pruebas o la ventana gráfica.

